TECHNICAL UNIVERSITY OF MOLDOVA
Faculty of Computers, Informatics, and Microelectronics
Department of Software Engineering
Software Engineering Study Programme

REPORT ON LABORATORY WORK NO. 10

OPERATING SYSTEMS

Add Your Own System Call: trace

Done by: st. gr. FAF-241, Lucian-Adrian Gavril

Checked by: univ. assist., Reițman Patricia

Chișinău, 2026

TABLE OF CONTENTS

INTRODUCTION 3

1 THE NUMBER AND THE USER SIDE 4

1.1 Four edits and the stub 4

1.2 The trace program 4

1.3 Check 1: the number reaches the kernel 5

2 THE KERNEL SIDE 6

2.1 The mask, the handler and the inheritance 6

2.2 The table, the names and the print 6

2.3 Build 7

2.4 Check 2: trace works 8

3 PLAYING WITH THE MASK 9

3.1 All calls 9

3.2 Only fork 9

3.3 Only write 10

3.4 Only open and close 11

3.5 Edge cases 11

4 OPTIONAL: SYSINFO 13

4.1 Kernel side 13

4.2 Build and run 14

CONCLUSIONS 16

INTRODUCTION

Purpose of the work. To change the kernel for the first time: to add a new system call, `trace`, to xv6 and follow its number, 23, from a user program through the `ecall` stub, the system-call table and the handler, down to a line in `syscall()` that prints every call the traced process makes. In Lab 9 `strace` did this from outside on Linux; here the same function is built inside the kernel. The optional part adds a second call, `sysinfo` (number 24), which copies a structure from the kernel to user memory.

Environment. Debian GNU/Linux 13, `riscv64-linux-gnu` GCC 14, QEMU 10.0. xv6 is the current upstream `mit-pdos/xv6-riscv`, in which 22 is already `sync` and the kernel prints with `printk`, with the programs of Labs 4–6 committed first as the base. Every change is in `lab10.patch` (13 files, 140 lines added, none removed), and the modified files are in `kernel/`, `user/` and `Makefile`. Each QEMU session started from a freshly built `fs.img`. Every terminal figure is the real output of the commands shown in it.

1 THE NUMBER AND THE USER SIDE

1.1 Four edits and the stub

Part 1 gives the call a number and lets user programs ask for it (Figure 1.1): `#define SYS_trace 23` in `kernel/syscall.h`, the declaration `int trace(int);` in `user/user.h`, `entry("trace");` in `user/usys.pl`, and `$U/_trace\` in `UPROGS`. At build time `usys.pl` generates `user/usys.S`, and the generated stub is exactly three instructions: `li a7, SYS_trace` puts 23 in register `a7`, `ecall` traps into the kernel, `ret` returns with the result in `a0`. This is the whole user side of a system call: the C function `trace()` is only a number and a trap.

```
dev1@dev1:~/xv6-riscv$ python3 /tmp/labcap10/lab10_apply.py . 1 && cp /tmp/labcap10/trace.c user/trace.c
part 1 applied
dev1@dev1:~/xv6-riscv$ git diff -U0 kernel/syscall.h user/user.h user/usys.pl Makefile | grep '^[-+@]' |
 grep -v '^+++\|^---'
@@ -149,0 +150 @@ UPROGS=\
+       $U/_trace\
@@ -23,0 +24 @@
+#define SYS_trace  23
@@ -27,0 +28 @@ int sync(void);
+int trace(int);
@@ -45,0 +46 @@ entry("sync");
+entry("trace");
dev1@dev1:~/xv6-riscv$ make -s fs.img >/dev/null 2>&1 ; grep -A3 '^trace:' user/usys.S
trace:
 li a7, SYS_trace
 ecall
 ret
```

Figure 1.1 – Part 1 edits and the generated stub in usys.S

1.2 The trace program

`user/trace.c` is the program from the brief (Figure 1.2). It sets the mask on its own process and then calls `exec`, which replaces the program but keeps the process, so the mask stays on the command that runs next. If `exec` returns at all, it failed.

```
dev1@dev1:~/xv6-riscv$ cat user/trace.c
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if (argc < 3) {
    fprintf(2, "usage: trace mask command [args...]\n");
    exit(1);
  }
  if (trace(atoi(argv[1])) < 0) {
    fprintf(2, "trace: failed\n");
    exit(1);
  }
  exec(argv[2], argv + 2);
  fprintf(2, "trace: exec %s failed\n", argv[2]);
  exit(1);
}
```

Figure 1.2 – user/trace.c

1.3 Check 1: the number reaches the kernel

With only Part 1 done, the kernel has no entry for 23 and prints exactly the expected complaint (Figure 1.3): `3 trace: unknown sys call 23`, then `trace: failed` because the call returned -1. The message comes from the `else` branch of `syscall()`, so it proves that the program executed `ecall` with `a7` = 23 and that the kernel read the number.

```
xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ trace 32 grep hello README
3 trace: unknown sys call 23
trace: failed
```

Figure 1.3 – Check 1: unknown sys call 23

2 THE KERNEL SIDE

2.1 The mask, the handler and the inheritance

Each process keeps its own mask in `struct proc` (`int trace_mask;` after `name`). The handler `sys_trace` reads argument 0 with `argint` (the value the user passed in register `a0`) and stores it on the calling process. In `fork()` the child copies the parent's mask right after the name. Figure 2.1 shows these three changes as a diff against the base.

```
dev1@dev1:~/xv6-riscv$ git diff -U1 kernel/proc.h kernel/sysproc.c kernel/proc.c | grep -v '^diff\|^inde
x\|^---\|^+++'
@@ -290,2 +290,3 @@ kfork(void)
   safestrcpy(np->name, p->name, sizeof(p->name));
+  np->trace_mask = p->trace_mask;

@@ -103,2 +103,3 @@ struct proc {
   char name[16];               // Process name (debugging)
+  int trace_mask;              // which system calls to print
 };
@@ -112 +112,10 @@ sys_uptime(void)
 }
+
+uint64
+sys_trace(void)
+{
+  int mask;
+  argint(0, &mask);
+  myproc()->trace_mask = mask;
+  return 0;
+}
```

Figure 2.1 – proc.h, sysproc.c and proc.c changes

2.2 The table, the names and the print

In `kernel/syscall.c` the handler is declared and placed at index 23 of `syscalls[]`, which is what turns "unknown sys call 23" into a real call (Figure 2.2). The kernel only knows numbers, so a table `syscallnames[]` placed above `syscall()` maps each number to its name. The print is added right *after* `p->trapframe->a0 = syscalls[num]();`, so the return value is already in `a0`:

```
if ((p->trace_mask >> num) & 1) printk("%d: syscall %s -> %ld\n", ...);
```

```
dev1@dev1:~/xv6-riscv$ git diff -U2 kernel/syscall.c | grep -v '^diff\|^index\|^---\|^+++'
@@ -104,4 +104,5 @@ extern uint64 sys_mkdir(void);
 extern uint64 sys_close(void);
 extern uint64 sys_sync(void);
+extern uint64 sys_trace(void);

 // An array mapping syscall numbers from syscall.h
@@ -131,7 +132,19 @@ static uint64 (*syscalls[])(void) = {
   [SYS_close]   = sys_close,
   [SYS_sync]    = sys_sync,
+  [SYS_trace]   = sys_trace,
   // clang-format on
 };

+static char *syscallnames[] = {
+  [SYS_fork] "fork",   [SYS_exit] "exit",     [SYS_wait] "wait",
+  [SYS_pipe] "pipe",   [SYS_read] "read",     [SYS_kill] "kill",
+  [SYS_exec] "exec",   [SYS_fstat] "fstat",   [SYS_chdir] "chdir",
+  [SYS_dup] "dup",     [SYS_getpid] "getpid", [SYS_sbrk] "sbrk",
+  [SYS_pause] "pause", [SYS_uptime] "uptime", [SYS_open] "open",
+  [SYS_write] "write", [SYS_mknod] "mknod",   [SYS_unlink] "unlink",
+  [SYS_link] "link",   [SYS_mkdir] "mkdir",   [SYS_close] "close",
+  [SYS_sync] "sync",   [SYS_trace] "trace",
+};
+
 void
 syscall(void)
@@ -145,4 +158,7 @@ syscall(void)
     // and store its return value in p->trapframe->a0
     p->trapframe->a0 = syscalls[num]();
+    if ((p->trace_mask >> num) & 1)
+      printk("%d: syscall %s -> %ld\n",
+             p->pid, syscallnames[num], p->trapframe->a0);
   } else {
     printk("%d %s: unknown sys call %d\n", p->pid, p->name, num);
```

Figure 2.2 – syscall.c: declaration, table entry, names and the print

2.3 Build

The tree at the commit with Parts 1 and 2 builds with exit status 0 (Figure 2.3). The kernel is compiled with `-Wall -Werror`, so any compiler warning in the new code would have stopped the build. The only two messages are the linker's standard notice about an RWX load segment for `kernel/kernel` and `user/_forktest`, which upstream xv6 prints in the same way before any change.

```
dev1@dev1:~/xv6-riscv$ git log --oneline -3
a4aa03c lab10 trace.c
16a58a0 lab10 trace
2c945b2 labs 4-6 programs
dev1@dev1:~/xv6-riscv$ make kernel/kernel fs.img > build2.log 2>&1 ; echo "make exit: $?"
make exit: 0
dev1@dev1:~/xv6-riscv$ grep 'warning:\|error:' build2.log
riscv64-linux-gnu-ld: warning: kernel/kernel has a LOAD segment with RWX permissions
riscv64-linux-gnu-ld: warning: user/_forktest has a LOAD segment with RWX permissions
dev1@dev1:~/xv6-riscv$ ls -l kernel/kernel user/_trace | cut -c25-
76680 Nov  2 17:49 kernel/kernel
34232 Nov  2 17:49 user/_trace
```

Figure 2.3 – Build of Parts 1 and 2: exit 0, only the upstream linker notices

2.4 Check 2: trace works

`trace 32 grep hello README` prints exactly the four required lines (Figure 2.4): `read -> 1023`, `965`, `453`, and `0` for the end of the file. $1023 + 965 + 453 = 2441$, and `ls README` in the same session shows the size 2441. `grep` reads with a 1024-byte buffer and keeps the unfinished last line at its start, so the reads return less than the buffer size each time.

```
xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ trace 32 grep hello README
3: syscall read -> 1023
3: syscall read -> 965
3: syscall read -> 453
3: syscall read -> 0
$ ls README
README          2 2 2441
```

Figure 2.4 – Check 2: read 1023, 965, 453, 0, and README is 2441 bytes

3 PLAYING WITH THE MASK

3.1 All calls

With all 31 bits on, `trace 2147483647 grep hello README` shows the whole life of the command (Figure 3.1): `trace -> 0` (the call that set the mask is itself traced, because the print runs after the mask is stored), `exec -> 3` (`argc` of `grep hello README`), `open -> 3` (the first free descriptor, as in Lab 9), the four reads, and `close -> 0`. `exit` is not printed: it never returns to the line after the call.

```
$ trace 2147483647 grep hello README
5: syscall trace -> 0
5: syscall exec -> 3
5: syscall open -> 3
5: syscall read -> 1023
5: syscall read -> 965
5: syscall read -> 453
5: syscall read -> 0
5: syscall close -> 0
```

Figure 3.1 – trace 2147483647: every system call of grep

3.2 Only fork

`trace 2 usertests forkforkfork` traces only `fork` (bit 1) on a test that forks until the process table is full (Figure 3.2). Pids 6 to 11 print lines, the returned pids grow up to 67, and then `fork -> -1`: the table has `NPROC` = 64 slots. The test ends with `OK` and `ALL TESTS PASSED`.

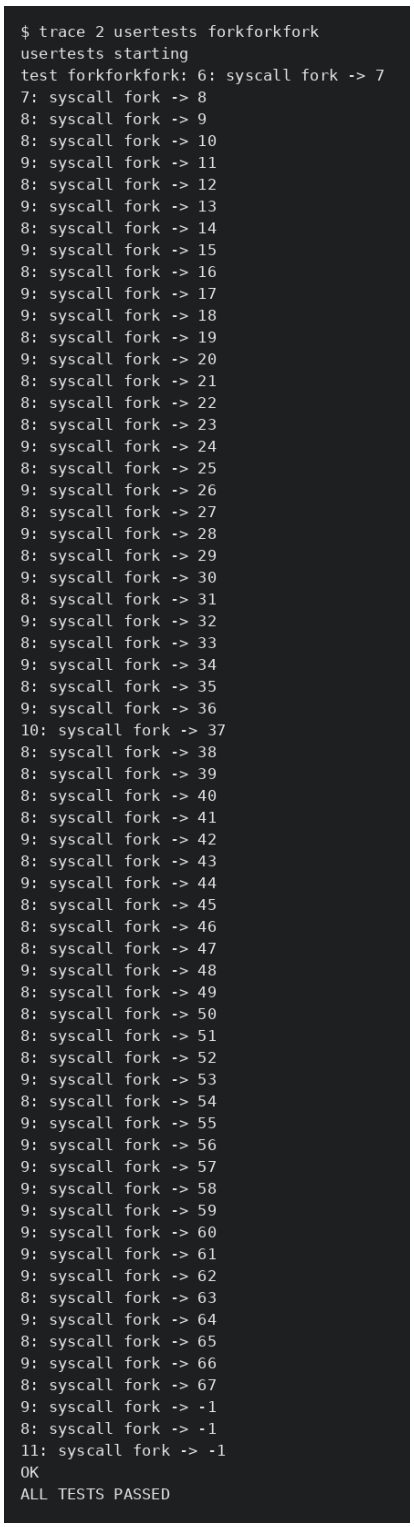


Figure 3.2 – trace 2 usertests forkforkfork

3.3 Only write

`trace 65536 echo hi` traces only `write` (bit 16), and the trace line appears between `hi` and the newline (Figure 3.3).

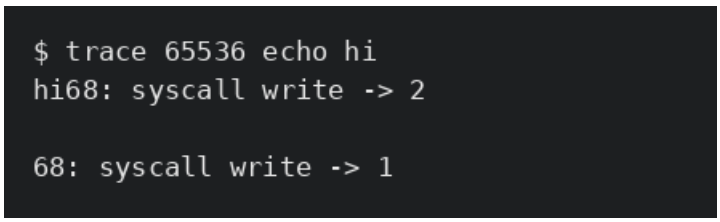


Figure 3.3 – trace 65536 echo hi

3.4 Only open and close

Mask $2^{15} + 2^{21} = 2129920$ selects `open` (15) and `close` (21) and nothing else (Figure 3.4). For `grep` it prints exactly one `open -> 3` and one `close -> 0`. In `cat README | wc` only `cat` is traced: the shell starts `trace` as the left side of the pipe, so `wc` is another child of the shell and never receives the mask.

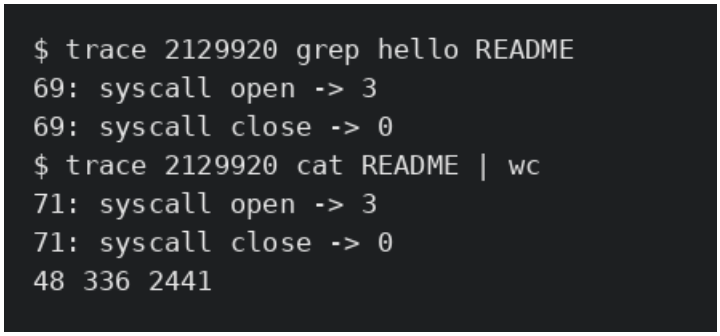


Figure 3.4 – trace 2129920: open and close only

3.5 Edge cases

Mask 0 prints nothing; `trace 32` without a command prints the usage line; a missing program reaches the `fprintf` after `exec` (Figure 3.5).

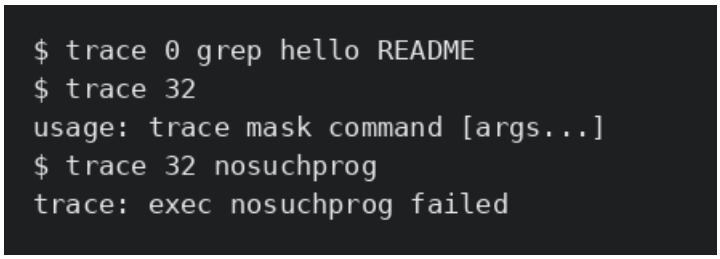


Figure 3.5 – Mask 0, missing command, missing program

Observe answers

1. In `trace 32 grep hello README` why do you see only read, and not open or close? 32 is 2^5: only bit 5 is set, and 5 is `SYS_read`. `syscall()` prints only when `(p->trace_mask >> num) & 1` is 1. For `open` (15) and `close` (21) that bit is 0, so they run normally and are not printed. Figure 3.1 shows that they are made.

2. In forkforkfork, several pids print lines. Which line of code from Part 2 makes that possible?

`np->trace_mask = p->trace_mask;` in `fork()` (`kernel/proc.c`). Without it a child would start with mask 0 from `allocproc`, and only the first process would print.

3. In `trace 65536 echo hi` the trace line is printed between "hi" and the newline. Why? How many times does echo call write? `echo` calls `write` twice: `write(1, "hi", 2)` for the argument and `write(1, "\n", 1)` for the end of the line. The kernel prints its trace line after each call returns, so the console receives `hi`, `68: syscall write -> 2`, the newline, then `68: syscall write -> 1`.

4. Which mask traces both open (15) and close (21), and nothing else? $2^{15} + 2^{21} = 32768 + 2097152 =$ **2129920**, verified in Figure 3.4.

4 OPTIONAL: SYSINFO

4.1 Kernel side

`sysinfo` (number 24) fills a `struct sysinfo { uint64 freemem; uint64 nproc; }` declared in the new `kernel/sysinfo.h` (Figure 4.1). Two helpers collect the values: `freemem()` in `kalloc.c` walks the free list under `kmem.lock` and multiplies the number of pages by `PGSIZE`; `nproc()` in `proc.c` counts the slots of `proc[]` whose state is not `UNUSED`. `sys_sysinfo` takes the user address with `argaddr` and copies the structure out with `copyout`, which fails for an address the process does not own, so the call returns -1 instead of writing into the kernel or crashing. The name is also added to `syscallnames[]`, so `trace` can show it.

```
dev1@dev1:~/xv6-riscv$ git diff -U0 kernel/ | grep '^[+@]' | grep -v '^+++'
@@ -61,0 +62 @@ void            kfree(void *);
+uint64          freemem(void);
@@ -90,0 +92 @@ int             killed(struct proc*);
+uint64          nproc(void);
@@ -82,0 +83,12 @@ kalloc(void)
+
+// Bytes on the free list (Lab 10, sysinfo).
+uint64
+freemem(void)
+{
+  uint64 n = 0;
+  acquire(&kmem.lock);
+  for (struct run *r = kmem.freelist; r; r = r->next)
+    n += PGSIZE;
+  release(&kmem.lock);
+  return n;
+}
@@ -704,0 +705,14 @@ procdump(void)
+
+// Number of process slots in use (Lab 10, sysinfo).
+uint64
+nproc(void)
+{
+  uint64 n = 0;
+  for (struct proc *p = proc; p < &proc[NPROC]; p++) {
+    acquire(&p->lock);
+    if (p->state != UNUSED)
+      n++;
+    release(&p->lock);
+  }
+  return n;
+}
@@ -106,0 +107 @@ extern uint64 sys_trace(void);
+extern uint64 sys_sysinfo(void);
@@ -134,0 +136 @@ static uint64 (*syscalls[])(void) = {
+  [SYS_sysinfo] = sys_sysinfo,
@@ -146 +148 @@ static char *syscallnames[] = {
+  [SYS_sync] "sync",   [SYS_trace] "trace", [SYS_sysinfo] "sysinfo",
@@ -24,0 +25 @@
+#define SYS_sysinfo 24
@@ -7,0 +8 @@
+#include "sysinfo.h"
@@ -121,0 +123,15 @@ sys_trace(void)
+
+uint64
+sys_sysinfo(void)
+{
+  uint64 addr;
+  struct sysinfo info;
+  struct proc *p = myproc();
+
+  argaddr(0, &addr);
+  info.freemem = freemem();
+  info.nproc = nproc();
+  if (copyout(p->pagetable, p->sz, addr, (char *)&info, sizeof(info)) < 0)
+    return -1;
+  return 0;
+}
dev1@dev1:~/xv6-riscv$ cat kernel/sysinfo.h
struct sysinfo {
  uint64 freemem;   // free memory, in bytes
  uint64 nproc;     // processes not UNUSED
};
```

Figure 4.1 – sysinfo: kernel side

4.2 Build and run

The full tree builds with exit status 0 (Figure 4.2). The test program `user/sysinfo.c` checks the values at four moments (Figure 4.3). At start there are 3 processes (`init`, `sh`, `sysinfo`). After `fork` there are 4, and 10 pages are gone: the child's trapframe, its page-table pages and the copy of the parent's memory (kernel stacks are allocated once at boot, so they do not count). `sbrk(40960)` takes exactly 10 more pages.

After `wait` the child's 10 pages come back and the count returns to 3. A pointer outside the process returns −1. Run under `trace 16777216` (bit 24), every call is printed with its result, and the last one is `sysinfo -> -1`.

```
dev1@dev1:~/xv6-riscv$ make kernel/kernel fs.img > /tmp/labcap10/build3.log 2>&1 ; echo "make exit: $?"
make exit: 0
```

Figure 4.2 – Build of the full tree

```
xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ sysinfo
start      : free 133251072 bytes (32532 pages), 3 processes
after fork : free 133210112 bytes (32522 pages), 4 processes
after sbrk : free 133169152 bytes (32512 pages), 4 processes
after wait : free 133210112 bytes (32522 pages), 3 processes
bad pointer: sysinfo returned -1
$ trace 16777216 sysinfo
5: syscall sysinfo -> 0
start      : free 133251072 bytes (32532 pages), 3 processes
5: syscall sysinfo -> 0
after fork : free 133210112 bytes (32522 pages), 4 processes
5: syscall sysinfo -> 0
after sbrk : free 133169152 bytes (32512 pages), 4 processes
5: syscall sysinfo -> 0
after wait : free 133210112 bytes (32522 pages), 3 processes
5: syscall sysinfo -> -1
bad pointer: sysinfo returned -1
$ trace 32 grep hello README
7: syscall read -> 1023
7: syscall read -> 965
7: syscall read -> 453
7: syscall read -> 0
```

Figure 4.3 – sysinfo at start, after fork, sbrk and wait, a bad pointer, and traced with bit 24

CONCLUSIONS

A system call is a number agreed on by both sides. The user side of `trace` is three instructions that put 23 in `a7` and execute `ecall`; the kernel side is an index into `syscalls[]`. Check 1 showed that the number arrives before any kernel code exists for it, and Check 2 showed that one entry in the table is enough to connect it to a handler.

Where a line is placed matters as much as what it says. The print must come after the call to see the return value, which is why `trace -> 0` itself appears with mask 2147483647 and why `exit` never does. The single line in `fork()` decides whether tracing follows a process tree: with it, pids 6 to 11 printed their forks in `forkforkfork`, while in `cat README | wc` the `wc` process, which did not descend from `trace`, was not traced.

The mask is read in binary, one bit per call number: 32 is `read` alone, 65536 is `write`, 2129920 is `open` and `close`. The trace lines also expose program behaviour, such as `grep` reading README in three pieces and `echo` writing its word and its newline separately.

`sysinfo` added the part `trace` does not need: returning a structure through a user pointer. `copyout` is what makes that safe, and the bad-pointer test returned -1 instead of corrupting memory. The page counts matched the process lifecycle exactly: 10 pages for a forked child, 10 for `sbrk(40960)`, all returned after `wait`.